

Matthew Washburn

Dr. Nathan Gossett

COS 320

18 December 2025

3D Fluid Simulation Write-Up and Citations

I started my final project with a completed project four, so I already had object creation, driving physics, and lighting. My first step towards a 3D fluid simulation was to create one particle and enact the forces of gravity on it. I used my existing sphere object to create a new small blue particle sphere, and did a quick search to find the velocity of a falling object equation (1). I learned that velocity is dependent on the force of gravity scaled by the change in time. I then gave the particle object a position and a velocity attribute. Once I applied this equation, I started to dive into the world of collision physics. I started with simple barrier collision physics based on particle offsets in world space. I had already done this in project 3, so this was trivial. Next, I learned that collision physics was based on the normal vector created in the direction of the collision between the two objects (2). I started experimenting with this by applying gravity to my particle and dropping it, and once it hits the ground, I flip its velocity in the direction that the collision occurred, which, in this simple case, was just flipping its y velocity. This created a simple bouncy ball. I then focused on creating lots of particles. It was at this point that I talked to Dr. Gossett, and he helped me realize that using my very dense sphere object class for every particle's appearance and position would be very inefficient, so I created a much lighter raw data type for each particle and moved around a singular sphere object to each particle's positioning in rendering. A huge lesson I learned throughout this process was how important it was to code algorithms that are used many times every single frame as efficiently as possible. I was then able

to create a large number of particles on the screen at once. To position them in a cube shape, I found a simple algorithm online that allows you to turn 1D indices into a cube shape with equal distances apart. With this algorithm, I learned how to use modulo to step per particle in the x direction, per row in the y direction, and per layer in the z direction (3). Next, it was time to figure out particle-to-particle collisions so my particles could “see” one another. I learned that you first calculate the distance vector between the two particles, the minimum distance they should ever be to one another (double the radius), and if a distance vector is ever less than that minimum, you know you have a collision. Once you know the particles collided, you flip the velocity of each particle along the same collision normal if the particles are going perfectly towards each other, and then the energy transfer depends on the relative velocity. I also added a bounce coefficient to determine the loss of energy in each collision (2). Although I didn’t use this test particle collision function in my final fluid simulation, the knowledge was very useful to transfer over to car and wheel collision detection, and the test simulation was very useful for checking if the other collision detection methods worked. For my car collision and wheel collision functions, the collision detection method was pretty similar, except for some extra steps in accounting for car and wheel rotation with WebGL’s right-handed nature that I learned in project three, and accounting for the shape of the cylinder tires and car body (4, 5). After this, it was onto the actual particle physics. The algorithm for this function is based entirely on a position-based fluids algorithm from an academic research paper. At a high level, I learned that this algorithm is based on first predicting the next location of each particle based on its velocity and change in time, and then storing the predicted particle locations and performing the fluid simulations on those. Next, each particle finds the particles that are physically near it in world space, and then applies a smoothing kernel function that produces a density value for each

particle based on the influence that each neighboring particle has on it and vice versa. In addition to this, the change in density (kernel gradient) between every neighbor particle is calculated, and the density divided by the sum of the gradient gives you λ , which tells you the vector needed to apply to the particle that will give the particle the greatest increase or decrease in density to get as close as it possibly can to its rest density. This value is then used to calculate the change in position required for the set of particles one frame into the future, and then the next positions are finally ready to be swapped for the current positions, and then applied in the next render pass. This algorithm also depends on a higher solver iteration count (performing the simulation multiple times per frame) so that gravity doesn't smush the particles to the ground before they have a chance to be adjusted by the fluid simulation (6). Throughout much of this fluid simulation process, I used a very helpful video that taught me the high-level flow of what it takes to achieve fluid simulations. It helped me keep a clear path in mind of how to get from point A to point B (7). I also consulted additional academic sources to help me understand this algorithm (8, 9). Initially, I did this by iterating through every particle, but as I've said, one of my big lessons for this project was learning the importance of optimization. $O(n^2)$ fluid simulation time was not going to cut it. I did some research and came across a much more efficient method of neighbor search called spatial hashing. I learned in this method that if you create a 3D hashed grid with cell sizes larger than your particles, and then only have each particle search the set of $3 \times 3 \times 3$ grid cells around and including its current grid cell in a cube shape for neighbors, you take your simulation from $O(n^2)$ to $O(n * k)$, a significant improvement (10). Once I completed this step, I wanted to optimize further to increase performance. I had heard Dr. Gossett mention instance rendering, so I decided to do more research. Essentially, this method improves performance by reducing the number of draw calls from n particles to 1. It works by

creating an instance buffer to store the position of each particle and then sends that off to the GPU. The GPU can then do the position calculation on each particle and draw them all at the same time with one call (11, 12). For my HTML, I learned how to create pretty sliders to change my constants from a helpful website and simulation (13). After this, I went through the long haul of screen space fluid rendering. I learned how to draw to textures from the render to texture in-class activity using a surface and outputting a color into a frame buffer attached to a texture (14). I learned how to turn data into colors with my texture density map (15, 16). I learned that applying a bilateral filter required not only a density map but also the base color values of the texture in the same color output, using the alpha channel to store the density (16, 17). I learned that it is way more efficient to apply a Gaussian blur in two 1D passes rather than a 2D pass (18, 19). I learned how to use inverse view and projection matrices to move objects from clip space back to view space and then back to world space (20). This pipeline was a very intricate and difficult concept to understand, but after I understood what was happening, everything clicked. I got stuck before remembering a very important lesson that I once heard somewhere, but completely forgot about until now: “Understand to code, don’t code to understand.” It’s not a waste of time to learn the concepts first; it actually saves you a lot of time. If you don’t understand the concept of what you’re trying to code, it’s really hard to understand while you’re coding it. Once I discovered and understood the high-level concept of screen space fluid rendering, the rest was easy. Overall, I feel like I got a very well-rounded understanding of computer graphics after completing this project, which I’m very thankful for. I really got a lot out of this class. Thank you, Dr. Gossett!

References

1. <https://www.statisticshowto.com/calculus-problem-solving/velocity-of-a-falling-object/#:~:text=Back%20to%20Top-,1,a%20Falling%20Object%20Using%20Calculus>
2. <https://research.ncl.ac.uk/game/mastersdegree/gametechnologies/physicstutorials/4collisiondetection/Physics%20-%20Collision%20Detection.pdf>
3. <https://ennogames.com/blog/3d-and-2d-coordinates-to-1d-indexes>
4. https://www10.cs.fau.de/publications/theses/2010/Suenkel_BT_2010.pdf
5. https://learnwebgl.brown37.net/transformations2/transformations_rotate.html#:~:text=Angles%20have%20a%20direction.,Y%20maps%20Z%20to%20X
6. https://mmacklin.com/pbf_sig_preprint.pdf
7. <https://www.youtube.com/watch?v=rSKMYc1CQHE>
8. <https://www.cs.cmu.edu/~scoros/cs15467-s16/lectures/11-fluids2.pdf>
9. <https://courses.grainger.illinois.edu/CS418/sp2023/text/sph.htm>
10. <https://matthias-research.github.io/pages/tenMinutePhysics/11-hashing.pdf>
11. <https://www.youtube.com/watch?v=Ude1zZbf20s>
12. <https://github.com/scriptfoundry/WebGL2-Videos-Materials/blob/main/13.instanceddrawing.js>
13. <https://codepen.io/gbopola/pen/ExpVomr>
14. <https://moodle.bethel.edu/mod/folder/view.php?id=3217837>
15. <https://www.youtube.com/watch?v=kOkfC5fLfgE>
16. https://developer.download.nvidia.com/presentations/2010/gdc/Direct3D_Effects.pdf
17. https://github.com/SebLague/Fluid-Sim/blob/main/Assets/Scripts/Rendering/ScreenSpace/Smoothing/Bilateral_1D/BilateralPass.hlsl

18. <https://bartwronski.com/2020/02/03/separate-your-filters-svd-and-low-rank-approximation-of-image-filters/#:~:text=While%20you%20pay%20the%20cost,naive%2C%20non%2Dseparable%20approach.>
19. <https://www.youtube.com/watch?v=IEvLyMwh1k8>
20. <https://github.com/SebLague/Fluid-Sim/blob/main/Assets/Scripts/Rendering/ScreenSpace/Shader/NormalsFromDepth.shader>